Kormat: Fine-Tuned Small-Language-Model Eligibility Screening with a From-Scratch KV Cache, Tiled Attention and Speculative Decoding on a CPU

Taha Ahmed
Roll no. [TODO: roll number], Section [TODO: section]
[TODO: department / university]
[TODO: email]

***Abstract*—We study criterion-level clinical-trial eligibility screening: given a patient note, a criterion type (inclusion or exclusion) and a criterion text, a model must return one of the expert label categories, including "not enough information" and "not applicable", together with supporting sentence IDs. We use the NCBI TrialGPT criterion annotations (1,015 annotations, 53 patients, patient-grouped 37/8/8 split, seed 42). A Qwen2.5-1.5B-Instruct model with a LoRA adapter, trained in a disclosed joint session on a shared GPU, is evaluated by us on a CPU laptop before and after adaptation. Beyond accuracy, we implement our own pre-allocated KV-cache decoder, a tiled online-softmax attention, and a greedy speculative decoder with a Qwen2.5-0.5B-Instruct draft, and wrap everything in a browser application. On the held-out test patients the adapted model reaches [TODO: macro-F1 / accuracy]; the cache gives [TODO: x] speed-up, tiled attention changes peak memory by [TODO: y], and speculative decoding gives [TODO: z] with an acceptance rate of [TODO: a]. We report these numbers with their limits.**

***Index Terms*—clinical trial eligibility, criterion-level matching, LoRA, KV cache, online-softmax attention, speculative decoding, CPU inference**

I. Introduction and Problem Statement

Recruiting patients for clinical trials depends on reading long free-text notes against lists of inclusion and exclusion criteria. TrialGPT showed that a large language model can do this criterion by criterion, with explanations, close to expert accuracy [1]. That system relies on a closed, remote model, which is a cost and a privacy problem for patient text [2]. This work asks a narrower and more practical question: *how far can a small open model (1.5B parameters) adapted with parameter-efficient fine-tuning get on the criterion-level task, and what does it take to run it on a laptop CPU without a GPU?*

Problem statement. Input: a patient note split into numbered sentences, a criterion type (inclusion or exclusion) and a criterion text. Output: a label from the criterion type's own four-way set (included / not included / not enough information / not applicable for inclusion; excluded / not excluded / not enough information / not applicable for exclusion) and the list of supporting sentence IDs. Absent evidence is never treated as proof of eligibility or ineligibility. Any patient–trial verdict shown in our application is an aggregation by explicit rules, not a separately labelled ground truth.

Contributions.

1) A reproducible data pipeline over the mandated dataset with a patient-grouped split and leakage checks, excluding the dataset's GPT-4 predictions from all inputs (Sec. III).
2) A before/after evaluation of a LoRA-adapted Qwen2.5-1.5B-Instruct with accuracy, macro-F1, per-class scores, confusion matrices, evidence P/R/F1, split by criterion type (Sec. V). The adapter itself was trained in a *disclosed joint session* on a shared GPU with a classmate; evaluation and everything below are our own work.
3) Our own implementations of a pre-allocated KV-cache decoder, tiled online-softmax attention (Algorithm 1) and greedy speculative decoding (Algorithm 2), each validated against the reference implementation and benchmarked on CPU.
4) A browser application, "Kormat", serving the pipeline with cited evidence and rule-based aggregation (Fig. 7).

The remainder is organised as: related work (Sec. II), methods (Sec. III), experimental setup (Sec. IV), results (Sec. V), limitations and conclusion.

II. Related Work

Clinical eligibility reasoning. Early work framed patient-to-trial matching as retrieval, with test collections judged by medical professionals [3]. TrialGPT moved to criterion-level reasoning with GPT-4, reporting 87.3% criterion accuracy on the 1,015 expert-annotated pairs that form our dataset, and trial-level ranking [1]; its dependence on a closed model is the gap we address. Nievas et al. fine-tune open LLaMA models on GPT-4-generated synthetic data and report parity with proprietary models [2]; we instead train only on expert labels and never use GPT-4 outputs as features.

Architecture. The Transformer [4] is our attention reference. Grouped-query attention [5], used in Qwen2.5 [6], shrinks the KV cache (Qwen2.5-1.5B: 12 query, 2 KV heads), which is what makes a CPU cache practical.

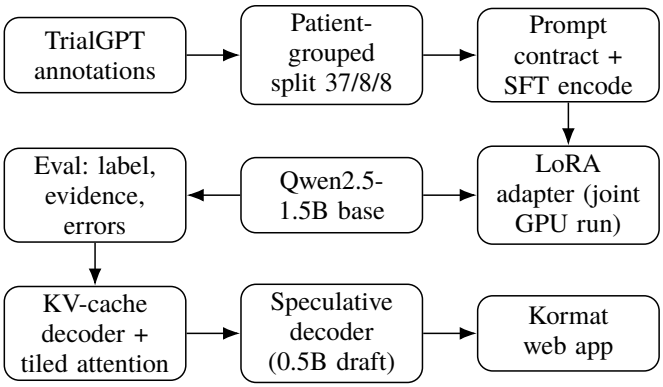


Fig. 1. System pipeline: data preparation, adapted model, evaluation, own inference stack, application.

Adaptation. LoRA trains low-rank updates on frozen weights [7]; QLoRA adds a 4-bit base [8]. For a 1.5B model, QLoRA's extra saving (about 2 GiB) does not justify its quantisation overhead, and full fine-tuning would need roughly 23 GiB (Sec. III-E).

Efficient inference. The online normaliser [9] and chunked attention [10] remove the need to materialise the $n \times n$ score matrix; FlashAttention [11], [12] makes it IO-aware on GPUs. Their speed-ups come from the GPU memory hierarchy, so we treat the CPU outcome as a hypothesis to measure, not a result to assume. Speculative decoding [13], [14] lets a small draft propose tokens verified by the large model in one pass; its gain depends on draft acceptance and on cheap verification. PagedAttention [15] and the scaling analysis of Pope et al. [16] address serving-scale KV management; our single-request app uses a contiguous pre-allocated cache instead.

III. Materials and Methods

A. System overview

Fig. 1 shows the pipeline. Components in the top row prepare data; the adapter was trained once in the joint session; everything in the lower row is ours and runs on CPU.

B. Dataset and split

We use only the NCBI TrialGPT criterion annotations [17]: 1,015 criterion annotations for 53 patients, each with a patient description, criterion type and text, expert label and expert evidence sentence IDs. The dataset also contains GPT-4 predictions and explanations; these are never used as input or in evaluation. Unique patient IDs are sorted, shuffled with `numpy.random.default_rng(42)`, and the first 37 go to training, the next 8 to validation and the last 8 to test; every annotation of a patient lies in exactly one split. The manifest is saved in `results/split_manifest.json` and a leakage check asserts empty pairwise patient intersections. The test split was untouched until final evaluation. **[TODO: generated table tables/data_stats missing]** Fig. 2 shows the label distribution.

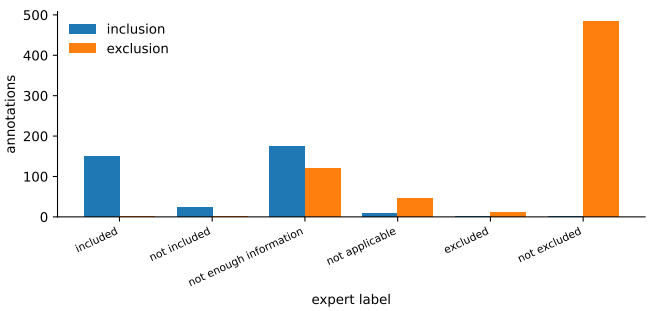


Fig. 2. Label distribution per criterion type and split.

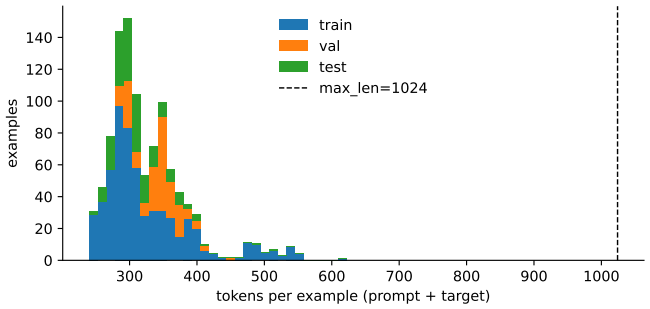


Fig. 3. Prompt length in tokens per split with the truncation threshold.

C. Input/output processing and tokenization

Each patient note is split into numbered sentences following the dataset's sentence indexing. A single prompt contract (system message, numbered note, criterion type, criterion, allowed labels, JSON return format) is shared by training, evaluation, benchmarks and the app, so all see the identical string. The target is a JSON object `{"label": ..., "evidence": [ids]}`; the label set depends on criterion type and is never collapsed to yes/no. Text is tokenised with the Qwen2.5 tokenizer (151,936 tokens, identical for the 1.5B and 0.5B models) through the chat template. Training examples longer than 1,024 tokens are dropped, never cut mid-text, so no relevant evidence is silently removed. **[TODO: generated table tables/token_lengths missing]** Fig. 3 shows the length distribution.

D. Model selection

Table I compares four open-weight decoder candidates using the `config.json` files on the Hugging Face Hub (Llama-3.2-1B is gated, so its figures come from a public mirror). We chose Qwen2.5-1.5B-Instruct [6]: 28 layers, hidden size 1536, 12 query heads and 2 KV heads with head dimension 128, 32,768-token context, Apache-2.0, and a KV cache of 28,672 bytes per token in bf16, 6.9× smaller than the multi-head SmolLM2-1.7B. Qwen2.5-0.5B-Instruct shares its tokenizer (identical vocabulary, merges, special tokens, chat template; zero id mismatches on 50 test strings) and serves as the draft model. The base was fixed by the shared training run; we

TABLE I
CANDIDATE OPEN DECODER MODELS (CONFIG.JSON; MEMORY BY ARITHMETIC, GIB).

	Qwen2.5-1.5B	Qwen3-1.7B	Llama-3.2-1B*	SmolLM2-1.7B
Params (B)	1.54	2.03†	1.24	1.71
Layers	28	28	16	24
Hidden	1536	2048	2048	2048
Heads Q/KV	12/2	16/8	32/8	32/32
head_dim	128	128	64	64
Vocab	151,936	151,936	128,256	49,152
Context	32,768	40,960	131,072	8,192
RoPE θ	10^6	10^6	5×10^5	130,000
Licence	Apache-2.0	Apache-2.0	Llama 3.2	Apache-2.0
fp32 / bf16 / 4-bit	5.75/2.88/0.72	7.57/3.78/0.95	4.60/2.30/0.58	6.38/3.19/0.80
KV B/token (bf16)	28,672	114,688	32,768	196,608

* gated; values from public mirror. † safetensors total incl. embeddings (card: 1.7B).

did not fine-tune the other candidates and make no claim that Qwen2.5 beats them.

E. Fine-tuning

Disclosure. The LoRA adapter was trained in a *joint session* on a shared GPU with M. Ruhan Kamran (23i-0062) and published as `arceus1rk/eligitrace-qwen2.5-1.5b-lora`. We did not train it alone; we use it, and all evaluation, inference code and application are our own, run on a CPU laptop.

Method choice. For $N = 1.54 \times 10^9$ parameters, full fine-tuning with mixed-precision AdamW needs about $16N \approx 23$ GiB for weights, gradients and optimiser state; LoRA [7] with r=16 on q, k, v, o, gate, up, down trains 18,464,768 parameters (1.2%), about 3.1 GiB in total; QLoRA [8] would reach about 1.0 GiB but adds quantisation overhead unnecessary at this size. Prompt-only is our "before" baseline.

Configuration. LoRA $r = 16$, $\alpha = 32$, dropout 0.05, 3 epochs, learning rate 2×10^{-4} with cosine schedule, effective batch size 16, bf16. The training objective is next-token cross-entropy on the assistant JSON only (prompt tokens masked).

F. Own KV-cache decoder

We implement a per-layer pre-allocated buffer of shape (batch, KV heads, max length, head dim) and one integer `length`. Prefill writes all prompt keys and values (already RoPE-rotated) once; each decode step writes one position and attends to the view `buf[..., :length, :]`. Compared with concatenating tensors each step (copy cost $O(n)$ per token), a slice write is $O(1)$ in cache size, and rollback needed by speculative decoding only moves the integer (Fig. 4). Cache size follows $2 \cdot L \cdot n_{kv} \cdot d_h \cdot b$ bytes per token.

G. Tiled online-softmax attention

For one head, $O = \text{softmax}(QK^\top/\sqrt{d})V$. Following the online normaliser [9] and its tiled use [10], [11], we iterate over key blocks keeping a running row maximum m, normaliser l and unnormalised output O (Fig. 5, Algorithm 1). The $L \times L$ matrix is never formed; only $B_q \times B_k$ tiles are. Accumulation is in fp32. Masks (causal, padding) are applied per tile; fully masked rows return zero and guard against $-\infty - (-\infty)$. We

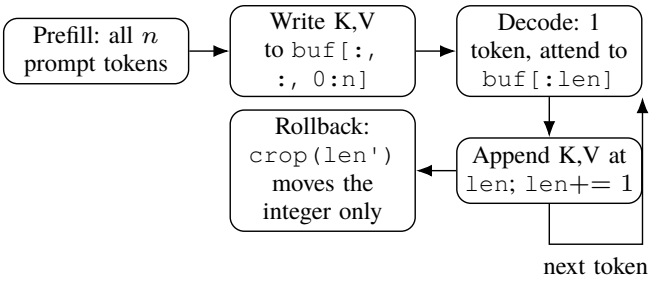


Fig. 4. Prefill and decode flow of the pre-allocated KV cache, with O(1) rollback.

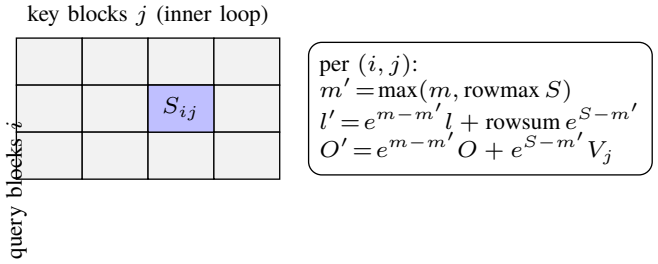


Fig. 5. Tile loop: for each query block, scan key blocks and update (m, l, O).

validate against the reference softmax attention (max absolute difference, and equality of generated tokens), see Sec. V. Because the speed-ups reported for this family of methods come from GPU memory hierarchy [11], we expect memory savings on CPU but not necessarily speed-ups, and test this.

H. Greedy speculative decoding

We implement greedy speculative decoding [13], [14]: the 0.5B draft proposes k tokens, the 1.5B target scores prefix plus proposals in a single forward pass, and the longest prefix on which draft and target argmax agree is accepted; the target's own next token is appended. Both caches roll back to the accepted length (Fig. 6, Algorithm 2). With greedy acceptance the output is token-identical to greedy decoding of the target, which we verify.

I. Application workflow

"Kormat" is a FastAPI service with a static browser front end. A user pastes a patient description and one or more criteria; the server splits sentences, builds the shared prompt, decodes (with cache and optional speculation), parses and validates the JSON (cited IDs must exist in the note), and returns per-criterion label, cited IDs, evidence text and an explanation based only on the note (Fig. 7). Multiple criteria may be combined into a patient–trial summary by explicit rules, which is an aggregation and not a ground-truth outcome.

IV. EXPERIMENTAL SETUP AND METRICS

Hardware and software. All reported numbers were produced on a laptop with an Intel Core i5-10310U (4 cores, 8 threads), 15 GB RAM and no GPU, with Python 3.10, PyTorch

Algorithm 1 Tiled online-softmax attention (one head)

Require: $Q \in \mathbb{R}^{L_q \times d}$, $K, V \in \mathbb{R}^{L_k \times d}$, block sizes B_q, B_k, mask M

for each query block Q_i (B_q rows) **do**
  $m \leftarrow -\infty$, $l \leftarrow 0$, $O \leftarrow 0$
  for each key block (K_j, V_j) **do**
    $S \leftarrow Q_i K_j^\top / \sqrt{d}$; apply mask M_{ij}
    $m' \leftarrow \max(m, \text{rowmax}(S))$
    $P \leftarrow \exp(S - m')$
    $l \leftarrow e^{m-m'} l + \text{rowsum}(P)$
    $O \leftarrow e^{m-m'} O + PV_j$; $m \leftarrow m'$
  end for
  output rows $O_i \leftarrow O/l$
end for

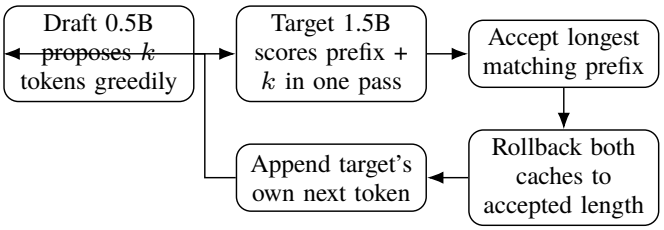


Fig. 6. Draft, verify and rollback loop of speculative decoding.

Algorithm 2 Greedy speculative decoding

Require: target T, draft D (same tokenizer), prompt x, draft length k, max new tokens N

prefill caches of T and D with x; $y \leftarrow [\,]$
while $|y| < N$ and no EOS **do**
  $d_1..d_k \leftarrow$ greedy decode of D (each step uses D's cache)
  $t_0..t_k \leftarrow$ arg max of T logits on $[\text{last}, d_1..d_k]$ (one pass)
  $a \leftarrow$ largest $a \le k$ with $d_i = t_{i-1}$ for all $i \le a$
  append $d_1..d_a$ and t_a to y
  roll back caches of T and D to the verified prefix (discard rejected positions)
end while
return y

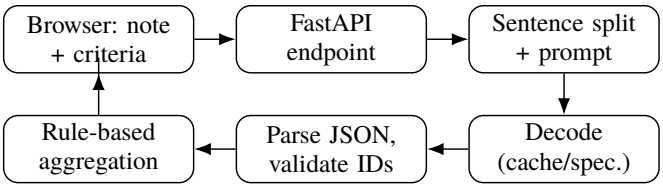


Fig. 7. Application request flow.

2.5.1 (CPU), Transformers 4.46.3 and PEFT 0.13.2, all pinned in the repository lock file. Only one large model is loaded at a time (a file lock enforces this). Each run writes a record (config, environment, metrics) under `results/<area>/`, and the commit SHA is stored with it.

Compared systems. (i) the base Qwen2.5-1.5B-Instruct with the same prompt contract (*before*); (ii) base plus the shared LoRA adapter (*after*). Both are evaluated on identical validation and test examples with greedy decoding (temperature 0), the same maximum number of new tokens and the same JSON parser. Invalid or unparsable outputs are counted as an explicit INVALID class and are never dropped. Hyperparameters and prompts were chosen using validation only; the test split was evaluated once, at the end.

Classification metrics. Accuracy, macro-F1, per-class precision/recall/F1 and confusion matrices, overall and separately for inclusion and exclusion criteria, each over its own four-way label set; macro-F1 averages over classes present in the gold labels. Labels are never collapsed to binary.

Evidence metrics. For each prediction, precision, recall and F1 of the predicted sentence-ID set against the expert set. Records where the gold evidence set is empty (e.g. "not enough information") are scored separately: a prediction with an empty set counts as correct, a non-empty set as a false positive. We report micro and per-record macro averages and the number of such records.

Systems metrics. Latency per generated token and end-to-end per criterion (median and spread over repeated runs after warm-up), peak resident memory, speed-up of cache vs. recomputation, peak memory and time of tiled vs. reference attention across sequence lengths, and for speculative decoding the acceptance rate, mean accepted tokens per verify pass and wall-clock speed-up over target-only greedy decoding. Output equality with the baseline is checked token by token.

Uncertainty. With 8 test patients, metric differences are noisy; we report counts alongside percentages and, where available, bootstrap intervals over patients.

V. Results and Discussion

A. Fine-tuning: training curves and validation

[TODO: generated table tables/finetune_summary missing]

B. Before/after on identical examples

[TODO: generated table tables/eval_before_after missing]

C. Evidence retrieval

[TODO: generated table tables/evidence_prf missing]

D. Error analysis

E. KV cache

[TODO: generated table tables/kvcache missing]

F. Tiled attention

[TODO: generated table tables/attention missing]

G. Speculative decoding

[TODO: generated table tables/speculative missing]

H. Application

VI. Limitations

- **Small data.** 53 patients, 8 in test; confidence intervals are wide and per-class results for rare labels rest on few examples.
- **Shared adapter.** The adapter was trained in a joint GPU session; we neither re-trained it independently nor ran seeds or ablations of the training, so we cannot separate training-recipe effects from base-model effects. The other candidate models were compared by configuration arithmetic only.
- **Criterion-level labels only.** The dataset has no patient–trial ground truth; our aggregation is rule based and unvalidated against experts.
- **Length handling.** Over-long examples were dropped from training (never cut); at inference very long notes can exceed the budget and evidence may be lost. **[TODO: state counts from the token-length analysis]**
- **CPU-specific systems results.** Tiled attention and speculative decoding are known GPU-driven speed-ups; our timings are for one laptop CPU with eager PyTorch and do not generalise to GPUs or other CPUs.
- **Not clinical software.** The system is a research prototype; outputs must not be used for real patient decisions.

VII. Conclusion

We built an open, locally runnable criterion-level eligibility screener around a LoRA-adapted Qwen2.5-1.5B-Instruct, evaluated it against its unadapted base on identical held-out patients, and implemented our own KV-cache decoder, tiled attention and speculative decoder with measurements on a CPU. **[TODO: one-sentence headline result and one-sentence systems finding, after Results are final.]** Future work: independent multi-seed training, larger external evaluation, and GPU benchmarks of the same code.

Availability

Source code: **[TODO: GitHub URL (TahaGPT/Kormat)]**. Adapter weights: Hugging Face `arceus1rk/eligitrace-qwen2.5-1.5b-lora`. Split manifest and logs are in the repository under `results/`. Demo video: **[TODO: video link]**.

References

[1] Q. Jin, Z. Wang, C. S. Floudas, F. Chen, C. Gong, D. Bracken-Clarke, E. Xue, Y. Yang, J. Sun, and Z. Lu, "Matching patients to clinical trials with large language models," *Nature Communications*, vol. 15, p. 9074, 2024.

[2] M. Nievas, A. Basu, Y. Wang, and H. Singh, "Distilling large language models for matching patients to clinical trials," *arXiv preprint arXiv:2312.09958*, 2023.

[3] B. Koopman and G. Zuccon, "A test collection for matching patients to clinical trials," in *Proc. 39th Int. ACM SIGIR Conf. on Research and Development in Information Retrieval*, 2016, pp. 669–672.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, arXiv:1706.03762.

[5] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, arXiv:2305.13245.

[6] Qwen Team, A. Yang, B. Yang, B. Zhang, B. Hui *et al.*, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.

[7] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Int. Conf. on Learning Representations (ICLR)*, 2022, arXiv:2106.09685.

[8] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems*, 2023, arXiv:2305.14314.

[9] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," *arXiv preprint arXiv:1805.02867*, 2018.

[10] M. N. Rabe and C. Staats, "Self-attention does not need $O(n^2)$ memory," *arXiv preprint arXiv:2112.05682*, 2021.

[11] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems*, 2022, arXiv:2205.14135.

[12] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *Int. Conf. on Learning Representations (ICLR)*, 2024, arXiv:2307.08691.

[13] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proc. Int. Conf. on Machine Learning (ICML)*, 2023, arXiv:2211.17192.

[14] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023.

[15] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. ACM Symp. on Operating Systems Principles (SOSP)*, 2023, arXiv:2309.06180.

[16] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," in *Proc. Machine Learning and Systems (MLSys)*, 2023, arXiv:2211.05102.

[17] NCBI, "TrialGPT-Criterion-Annotations dataset," Hugging Face Datasets, https://huggingface.co/datasets/ncbi/TrialGPT-Criterion-Annotations, 2024.

Appendix A
Use of AI Tools

Tools. This project was planned, coded and drafted with Claude Code (a lead agent, Claude Opus 5.5, coordinating specialist agents running Claude Sonnet 5.5) working in the repository under the author's direction. AI assistance covered task planning, literature search and drafting, source code, tests, the web application, LaTeX and figure scripts.

Verification. Every bibliographic entry was opened on its arXiv, publisher or Crossref page before inclusion; model facts come from the Hugging Face `config.json` files and model cards; all reported numbers are produced by scripts that write run records with the commit SHA, not typed by hand; implementations are checked against reference ones (attention against softmax attention, speculative output against greedy output) by tests; the split is checked for patient leakage. **[TODO: the author states which outputs they personally reviewed, ran and can explain.]**

Joint training disclosure. The LoRA adapter (`arceus1rk/eligitrace-qwen2.5-1.5b-lora`) was trained in a joint session on a shared GPU with M. Ruhan Kamran (23i-0062), because the author's laptop has no GPU. It is a disclosed shared artefact. No code, results, figures or text were copied from the other student's repository.

Limits of AI use. AI-generated text and code may contain errors; claims in this report are limited to what the cited sources and our measurements support.